The German Tank Problem

Team Members: _____ _____ _____

_____ _____

Background — read this before class

Through 1940 and 1941, the Allies badly needed to know how many tanks Germany was building, and had two ways of finding out. The first was conventional intelligence: spies, prisoner interrogations, intercepted signals, aerial reconnaissance. The second was a group of economists in London working with a much stranger kind of evidence — the serial numbers stamped on captured German equipment. Gearboxes, engines, chassis, and road wheels all carried numbers, and the Germans, being methodical, numbered them in sequence.

The idea is this. If you capture a handful of tanks at random and read their serial numbers, those numbers are a sample from the full run of production. The largest number you happen to see tells you something about how long that run must be. How much it tells you, and how to turn it into a number, is the whole problem.

Richard Ruggles and Henry Brodie published the wartime account in 1947. Postwar records let everyone check the answers:

Month	Conventional intelligence	Serial-number analysis	German records
June 1940	1,000	169	122
June 1941	1,550	244	271
August 1942	1,550	327	342

The spies were off by a factor of five. The statisticians were off by a few percent, and they were working from scrap metal. Just before D-Day the same method was turned on the Panther tank, whose numbers in northern France the Allied command had badly underestimated. Road wheels from two captured Panthers gave an estimate of 270 built in February 1944. German records, recovered after the war, put the true figure at 276.

Today you will reconstruct the method, find out how well it works, and find out how they knew how well it worked.

Part 1. Commit First

Your unit has captured five enemy tanks. Their serial numbers are:

```
62    214    389    605    880
```

Nobody knows how many the enemy has. You have to say something. Do this before you open the notebook.

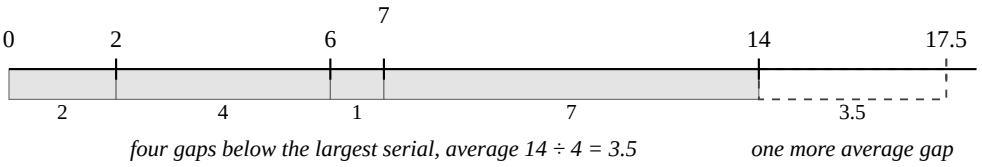
> **1.1 State your team's rule in words, then give the number it produces. Anyone should be able to apply your rule without asking what you meant.**
>
> **Estimate:** _____

1.2 Consider the simplest rule of all: report the largest serial number you saw. Give a reason it must be wrong — not "it feels low," but an argument for why it can never be too high.

Where the Correction Comes From

Before you write `est_corrected`, here is the argument behind it.

Your captured serial numbers chop the stretch from zero up to the largest one you saw into k pieces: the space below the smallest, and the space between each consecutive pair. If the capture was random, none of those gaps is special, so on average they are all the same size — namely m/k. Above the largest serial there is one more stretch, the tanks you never saw, and no reason it should be systematically wider or narrower than the others. So estimate it at one more average gap.



Above the line are the four serial numbers you captured; below it are the sizes of the gaps they create. Adding one more average gap gives $m + m/k = m(1 + 1/k)$, or 17.5 here.

The -1 in the formula is a counting correction. Serial numbers are whole numbers and the gaps include their endpoints, so without it the estimate lands one tank high on average. That brings this sample to 16.5 — the value your function has to return in 2.2.

Two checks worth making. If you captured every tank, then $k = N$ and $m = N$, and the formula returns exactly N. If you captured only one, it tells you to double the serial number — which is what you would have guessed anyway, since a single random serial sits halfway up the run on average.

Part 2. One Capture

Open `Class11_German_Tank_Problem_Skeleton.ipynb` and save it under your team's name.

2.1 Four more draws of five tanks from a fleet of 1000. Largest serial number each time:

_____ _____ _____ _____

2.2 Tested on `make_array(2, 6, 7, 14)`, your two functions must return exactly 14 and 16.5. Check both before going on. Do they? YES NO — fix them now.

2.3 Now the five real serial numbers.

`est_max` _____ `est_corrected` _____

2.4 Here is a third rule, which you will not be coding. Take twice the average of the serial numbers, minus one. Work it out by hand for your five tanks: _____

That answer is smaller than a serial number you have seen with your own eyes. Could it be the true number of tanks? This rule is nevertheless correct on average. What does that tell you about the difference between a rule being right on average and a rule being sensible every time?

Part 3. A Thousand Parallel Wars

You cannot tell whether a rule is good from one capture, because you get one capture and you never learn the answer. So invent an enemy whose size you already know — 1000 tanks — and fight the same war a thousand times.

3.1 Sketch both histograms, on the same bins. Mark 1000 on each one.

`max`	`corrected`

3.2 Which histogram is centered on the true value of 1000? _____

Are they symmetric, or do they have a long tail on one side?

_____

3.3 The serial numbers themselves are perfectly uniform — every number from 1 to 1000 is equally likely, and that distribution has no skew at all. Yet your histograms are lopsided. Where did the skew come from?

Part 4. Which Estimator Would You Take Into a War?

Fill in the table from your simulation of 1000 captures. The true value is 1000.

	mean	median	SD	IQR	within 20%
`max`					
`corrected`					

> **4.1 One estimator is wrong in the same direction every single time. Which one, in which direction, and by roughly how much? Explain why it could not have come out any other way.**

> **4.2 On a skewed distribution the SD and the IQR tell you different things about the spread. Which would you rather quote to a general who has to plan around your estimate, and why?**

Part 5. Testing a Claim

Conventional intelligence insists the enemy has 1500 tanks. The largest serial number you captured was 880.

> **5.1 Out of 1000 simulated captures from a fleet of 1500, the fraction whose largest serial number came out at 880 or below: _____**

> **5.2 The general asks whether the 1500 figure is believable. Answer him in three sentences, using the number you just computed. Do not use any vocabulary you have not been taught yet.**

Before You Leave

> **The true size of the enemy fleet, announced at the end of class: _____**

> **Your Part 1 estimate was off by _____, your `est_corrected` estimate by _____. Was the method wrong?**

Written Response

Answer both before the next class and bring them with you.

> **R1. Your histograms in Part 3 show a thousand estimates from a thousand captures — but in the real war there was only ever one capture, and it either happened or it did not. So what are those histograms a picture of?**

> **R2. Last class, a coin split an estate fairly on average and almost never fairly in practice. Today, one of your estimators is right on average and rarely right exactly. Say what the two situations have in common, and what it means to trust a procedure whose individual results you cannot trust.**

Code You Will Need

This is a reference, not a solution. You still have to decide what goes where.

Setup — run this first, exactly as written:

```
from datascience import *
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
import collections as collections
import collections.abc as abc
collections.Iterable = abc.Iterable
```

Making and drawing from arrays:

```
np.arange(1, 1001)                          # the numbers 1 through 1000
make_array(2, 6, 7, 14)                     # an array of specific values
np.random.choice(serials, 5, replace=False) # capture 5 different tanks
len(sample)                                 # how many values are in an array
```

Summarizing an array:

```
max(sample)             np.mean(sample)          np.median(sample)
np.std(sample)          np.percentile(sample, 25)
np.mean(estimates >= 800)   # the fraction of values that satisfy a condition
```

Repeating something 1000 times and keeping the answers:

```
results = make_array()

for i in np.arange(1000):
    sample = np.random.choice(serials, 5, replace=False)
    results = np.append(results, max(sample))
```

Tables and histograms:

```
t = Table().with_columns('max', max_estimates, 'corrected',
corrected_estimates)

bins = np.arange(0, 2001, 50)
t.hist('max', bins=bins)
plt.title('Estimator: max');
```

Two rules for today. Run every loop with 10 repetitions and read the output before you run it with 1000 — a bug is obvious in ten numbers and invisible in a thousand. And test every function on values you can check by hand before you trust it with anything.